TestScene UI 반응형 및 깨짐 방지 구현 분석 보고서

기기별 다양한 해상도 및 노치/컷아웃 환경 대응을 위한 5단계 방어 아키텍처 상세 분석

분석 대상 씬	기준 해상도	스케일 매치 모드	분석 일자 / 작성자
Assets/Scenes/TestScene.unity	540 × 960 (Portrait 9:16)	Expand Mode (Mathf.Min)	2026. 10. 01 / Antigravity AI

📌 요약 (Executive Summary)

본 보고서는 `TestScene`에서 다양한 모바일 디바이스(스마트폰, 태블릿, 폴더블) 및 PC 환경에서 화면 비율과 해상도가 변동할 때 **UI 요소가 찌그러지거나 잘리는 현상(UI 깨짐)을 방지하기 위해 적용된 기술적 구현 메커니즘**을 코드와 씬 데이터 레벨에서 정밀 분석한 결과입니다.

분석 결과, 본 프로젝트는 단일 기법에 의존하지 않고 ① **CanvasScaler(Expand 모드) 기반 전역 스케일링**, ② **SafeArea 컴포넌트를 통한 물리 노치/펀치홀 실시간 동적 앵커링**, ③ **UI 모듈별 상대 앵커(Anchor) 및 피벗(Pivot) 체계**, ④ **Auto-Layout Group(Horizontal/Grid/ContentSizeFitter) 엔진**, ⑤ **TextMeshPro Auto-Sizing 및 뷰 상태 제어**로 이어지는 「**5중 반응형 방어 아키텍처**」를 유기적으로 구축하여 UI 무결성을 보장하고 있습니다.

1. 반응형 UI 5중 방어 아키텍처 개요

스마트폰 디바이스는 16:9, 18:9, 19.5:9, 20:9 등 매우 다양한 화면비를 가지며, 상단 노치(Notch), 펀치홀 카메라, 하단 제스처 바(Home Indicator) 등의 물리적 컷아웃 영역이 존재합니다. `TestScene`에서는 아래와 같은 5단계 계층 방어선을 통해 UI가 깨지거나 조작 불능 상태가 되는 것을 완벽히 차단합니다.

1단계: 전역 해상도 스케일링 — CANVASSCALER

기준 해상도(540×960) & Expand 모드를 적용하여 캔버스가 기준 크기보다 작아지지 않게 보장하며, 화면비 변동 시 UI가 잘려나가는 것을 원천 방지.

2단계: 물리 안전영역 동적 제어 — SAFEAREA.CS

`Screen.safeArea`를 정규화(0~1)하여 RectTransform 앵커에 실시간 적용. 회전 및 창 크기 변경 시 `Update()`에서 즉각 재연산.

3단계: 목적별 앵커/피벗 분할 — RECTTRANSFORM

HUD(Top-Left), 조작 버튼(Bottom-Left/Right), 서브패널(비례 스트레치 0~0.2) 등 컴포넌트 특성에 맞춘 정밀 앵커링 설계.

4단계: 자동 레이아웃 엔진 — LAYOUT GROUPS

`HorizontalLayoutGroup`, `GridLayoutGroup`, `ContentSizeFitter`를 조합하여 동적 슬롯 생성 시 겹침 현상 방지.

5단계: 요소 레벨 오버플로우 방지 및 뷰 상호 배타성 관리 — TMP & UITOGGLEMANAGER

텍스트 길이에 따라 폰트 크기가 자동 축소되는 `TextMeshProUGUI Auto-Sizing`과 상점/부화소 UI가 동시에 열려 화면이 뒤덮이는 것을 방지하는 `UIToggleManager` 상호 배타적 토글 로직 적용.

2. 계층별 상세 구현 분석

2.1 Canvas & CanvasScaler: Expand 스케일링 메커니즘

`TestScene`의 캔버스 설정은 모바일 세로 게임 환경에 최적화된 구성을 갖추고 있습니다.

프로퍼티	설정값	UI 깨짐 방지 기여도
Render Mode	`Screen Space - Overlay`	3D 원근 카메라(FOV 60)와 UI 좌표계를 분리하여 카메라 회전/이동/줌에 UI가 영향받지 않도록 독립성 유지
UI Scale Mode	`Scale With Screen Size` (1)	고정 픽셀이 아닌 화면 해상도 변화에 비례하여 UI 전체 크기가 자동 스케일링
Reference Resolution	`540 × 960` (Portrait 9:16)	모바일 세로 비율의 표준 런타임 기준 좌표계 정의
Screen Match Mode	`Expand` (1)	화면 비율이 길어지거나 넓어질 때 UI가 화면 밖으로 잘리지(Crop) 않도록 보장

> 💡 **왜 MatchWidthOrHeight 대신 'Expand' 모드를 채택했는가?**
>
> Unity 소스코드(CanvasScaler.cs)에 따르면 Expand 모드는 다음과 같이 스케일 팩터를 연산합니다:
> `scaleFactor = Mathf.Min(screenSize.x / 540f, screenSize.y / 960f);`
> 이 방식은 화면의 가로/세로 중 **더 좁은 비율에 맞춰 스케일을 결정**하므로, 캔버스의 가상 영역이 기준 해상도(540×960)보다 절대로 작아지지 않습니다. 최신 20:9 폰(세로로 긴 화면)이나 태블릿(가로로 넓은 화면)에서도 UI가 화면 밖으로 잘려나가지 않고 추가적인 여백을 확보하는 안전한 구조를 형성합니다.

2.2 SafeArea 컴포넌트: 물리 안전영역 동적 앵커링

`Assets/Game/Scripts/UI/Util/SafeArea.cs` 스크립트는 Canvas 바로 아래 최상위 컨테이너로 배치되어, 디바이스의 물리적 노치/컷아웃/홈바 영역을 감지하고 캔버스의 앵커 좌표를 실시간으로 재구성합니다.

```
using UnityEngine;

public class SafeArea : MonoBehaviour
{
    private RectTransform rectTransform;
    private Rect lastSafeArea;
    private Vector2Int lastScreenSize;
    private ScreenOrientation lastOrientation;

    void Awake()
    {
        rectTransform = GetComponent<RectTransform>();
        ApplySafeArea();
    }

    void Update()
    {
        // 화면 회전, 해상도 변경, 폴더블폰 접힘/펼침 실시간 감지
        if (lastSafeArea != Screen.safeArea ||
            lastScreenSize.x != Screen.width ||
            lastScreenSize.y != Screen.height ||
            lastOrientation != Screen.orientation)
        {
            ApplySafeArea();
        }
    }

    void ApplySafeArea()
    {
        Rect safeArea = Screen.safeArea;
        lastSafeArea = safeArea;
        lastScreenSize = new Vector2Int(Screen.width, Screen.height);
        lastOrientation = Screen.orientation;

        // 픽셀 좌표계를 0.0 ~ 1.0의 정규화된 앵커 좌표로 변환
        Vector2 anchorMin = safeArea.position;
        Vector2 anchorMax = safeArea.position + safeArea.size;

        anchorMin.x /= Screen.width;
        anchorMin.y /= Screen.height;
        anchorMax.x /= Screen.width;
        anchorMax.y /= Screen.height;

        rectTransform.anchorMin = anchorMin;
        rectTransform.anchorMax = anchorMax;
        rectTransform.offsetMin = Vector2.zero;
        rectTransform.offsetMax = Vector2.zero;
    }
}
```

> 🔍 **SafeArea 구현의 핵심 특징 분석**
>
> - **정규화 앵커 변환(Normalization)**: 기기마다 다른 물리 해상도 픽셀을 `0.0 ~ 1.0` 비율로 환산하여 CanvasScaler 스케일 팩터와 무관하게 정확한 안전 영역을 보정합니다.

- **완전 밀착 (Zero Offset)**: `offsetMin = offsetMax = Vector2.zero`를 적용하여 앵커 박스와 RectTransform의 경계를 1:1로 일치시킵니다.
- **런타임 실시간 반응성**: `Update()`에서 디바이스 방향(`Screen.orientation`), 해상도(`Screen.width/height`), 안전영역(`Screen.safeArea`)을 지속적으로 트래킹하여 **화면 분할(Split Screen), 폴더블폰 접힘/펼침, 앱 회전** 시에도 UI가 깨지지 않고 즉시 갱신됩니다.

2.3 RectTransform 앵커(Anchor) 및 피벗(Pivot) 체계

`SafeArea`의 자식으로 배치된 모든 주요 UI 요소들은 화면 모서리 및 영역별로 특화된 앵커링 전략을 취하고 있습니다.

GameObject 명칭	Anchor Min / Max	Pivot	배치 전략 및 깨짐 방지 원리
Score / Gold / Life / Phase	`(0, 1) / (0, 1)`	`(0.5, 0.5)`	**좌상단(Top-Left) 고정**: 상단 상태바 및 노치 아래에 밀착되어 항상 안정적인 위치에 정보 노출
Btn_Shop / Btn_Hat	`(0, 0) / (0, 0)`	`(0.5, 0.5)`	**좌하단(Bottom-Left) 고정**: 하단 홈바 위 좌측 모서리에 안정적으로 안착
PhaseButton	`(1, 0) / (1, 0)`	`(1, 0)`	**우하단(Bottom-Right) 고정**: 피벗을 (1, 0)으로 맞추어 우측 하단 여백 0에 완벽히 밀착
ShopPanel (프리팹)	`(0, 0) / (1, 0.2)`	`(0.5, 0.5)`	**가로 100% 스트레치 + 하단 20% 높이 비율 점유**: 기기 너비가 늘어나면 자동으로 패널 가로폭이 확장됨
BenchPanel (프리팹)	`(0, 0) / (1, 0)`	`(0.5, 0.5)`	**하단 가로 스트레치 + 고정 높이(100px)**: 하단 거치대 역할을 하며 좌우 화면 폭에 자동 대응
InvnetoryPanel (프리팹)	`(0, 0) / (1, 1)`	`(0.5, 0.5)`	**안전영역 전체 스트레치(Full stretch)**: 인벤토리 모달 창이 안전영역 전체를 균일하게 채움

2.4 자동 레이아웃 엔진 (Layout Groups & ContentSizeFitter)

정적 배치가 아닌 아이템 목록이나 슬롯 배열은 해상도나 아이템 개수 변경 시 UI 요소가 겹치거나 밖으로 삐져나가기 쉽습니다. `TestScene`에서는 유니티의 자동 레이아웃 컴포넌트를 다음과 같이 적극 활용하고 있습니다.

패널 / 컨테이너	적용 컴포넌트	상세 설정값	깨짐 방지 동작 메커니즘
ShopPanel /SlotContainer	`HorizontalLayoutGroup`	• Child Alignment: Middle Center (4) • Force Expand W/H: True • Control Child W/H: False	3개의 상점 상품 슬롯이 패널 내에서 중앙에 고르게 정렬되며, 화면 폭이 넓어져도 슬롯 간격이 균일하게 확장되어 슬롯 겹침 현상 방지
BenchPanel /SlotContainer	`HorizontalLayoutGroup`	• Child Alignment: Middle Center (4) • Force Expand W/H: True	6개의 벤치 슬롯이 가로 폭 전체에 걸쳐 중앙 정렬 및 균등 분할되어 화면비가 바뀌어도 일정한 배치 유지
InvnetoryPanel /SlotContainer	`GridLayoutGroup`	• Cell Size: (55, 55) • Spacing: (5, 5) • Child Alignment: Middle Center • Constraint: Flexible (0)	화면 너비에 맞춰 격자형으로 슬롯이 자동 줄바꿈(Flexible Wrapping)되어 화면 해상도가 달라져도 슬롯이 잘리지 않음
ObjectStoragePanel /Content	`GridLayoutGroup` `ContentSizeFitter`	• Cell Size: (100, 100) • Constraint: Fixed Row Count (1) • **Horizontal Fit: Preferred Size (2)**	보관된 오브젝트 개수에 맞춰 가로 길이가 **동적으로 자동 확장**되며, `ScrollRect` 및 `Mask`와 연동되어 내용이 넘칠 때 영역 밖을 클리핑하고 부드러운 스크롤 제공

2.5 미세 레벨 방어: TextMeshPro Auto-Sizing & 뷰 배타성 제어

TextMeshPro Auto-Sizing — 텍스트 넘침 방지

Scene 내 대다수 TMP 컴포넌트(`Score`, `Gold`, `Life`, `Phase`, `Btn_Shop`, `Btn_Hat` 등)에 `enableAutoSizing = 1`, `fontSizeMin = 1`, `fontSizeMax = 72`가 지정되어 있습니다.

효과: 골드 수치가 수만~수십만 단위로 폭증하거나 디바이스 스케일링으로 텍스트 영역이 축소되더라도 폰트 크기가 자동으로 축소되어 **텍스트가 박스 밖으로 삐져나가거나 줄바꿈되어 깨지는 현상을 차단**합니다.

UIToggleManager 상호 배타성 제어 — 뷰 겹침 방지

`UIToggleManager.cs`는 상점(`ItemShopView`)과 부화소(`HatcheryView`)를 제어할 때 `IToggleableView` 인터페이스를 활용합니다.

효과: 특정 창을 열 때 다른 창을 자동으로 닫는(`otherPanel?.Close()`) 상호 배타적 토글 로직을 구현하여 **두 개의 대형 패널이 동시에 화면에 렌더링되어 UI가 뒤엉키는 시각적 파손을 구조적으로 예방**합니다.

3. TestScene 전체 UI 계층 구조도 (UI Hierarchy Map)

다음은 `TestScene` 내부의 Canvas로부터 파생된 전체 RectTransform 및 UI 컴포넌트의 트리 구조와 배치 특성입니다:

```
📦 Canvas [Screen Space - Overlay / CanvasScaler: Expand (540x960)]
├── 📁 ObjectStoragePanel [Anchor: (0,0)~(1,1), Pos:(-3, 133), Size:(447.3, 276.5)] (Image, ObjectStorageUI)
│   └── 📁 Scroll View [Anchor: Center (0.5, 0.5), Size:(500, 200)] (ScrollRect, Image)
│       ├── 📁 Viewport [Anchor: Full Stretch (0,0)~(1,1)] (Mask, Image) *영역 외 클리핑*
│       │   └── 📄 Content [Anchor: Top-Stretch (0,1)~(1,1)] (GridLayoutGroup, ContentSizeFitter)
│       └── 📁 Scrollbar Horizontal (Scrollbar)
├── 🛡️ SafeArea [Anchor: (0,0)~(1,1) → Screen.safeArea 정규화 동적 치환] (SafeArea.cs)
│   ├── 📄 ShopPanel (Prefab) [Anchor: Bottom-Stretch (0,0)~(1, 0.2)] (ItemShopView)
│   │   └── 📄 SlotContainer (HorizontalLayoutGroup: MiddleCenter, ForceExpand)
│   ├── 📄 BenchPanel (Prefab) [Anchor: Bottom-Stretch (0,0)~(1, 0), PosY:173, H:100] (BenchView)
│   │   └── 📄 SlotContainer (HorizontalLayoutGroup: MiddleCenter, ForceExpand)
│   ├── 📄 InvnetoryPanel (Prefab) [Anchor: Full-Stretch (0,0)~(1,1)] (InventoryView)
│   │   └── 📄 SlotContainer (GridLayoutGroup: Cell(55,55), Spacing(5,5), Flexible)
│   ├── 📄 HatcheryPanel (Prefab) [Anchor: Full-Stretch (0,0)~(1,1), PosY:-200, SizeY:-400] (HatcheryView)
│   ├── 🔘 Btn_Shop [Anchor: Bottom-Left (0,0), Pos:(60, 10)] (Button, Image, TMP)
│   ├── 🔘 Btn_Hat [Anchor: Bottom-Left (0,0), Pos:(20, 10)] (Button, Image, TMP)
│   ├── 🔘 PhaseButton [Anchor: Bottom-Right (1,0), Pivot:(1,0)] (Button, Image, TMP)
│   ├── 📝 Score [Anchor: Top-Left (0,1), Pos:(45.3, -15)] (TextMeshProUGUI Auto-Sizing)
│   ├── 📝 Gold [Anchor: Top-Left (0,1), Pos:(15, -15)] (TextMeshProUGUI Auto-Sizing)
│   ├── 📝 Life [Anchor: Top-Left (0,1), Pos:(77.4, -15)] (TextMeshProUGUI Auto-Sizing)
│   └── 📝 Phase [Anchor: Top-Left (0,1), Pos:(108.8, -15)] (TextMeshProUGUI Auto-Sizing)
├── ⚙️ ToggleManager (UIToggleManager) *상점/부화소 상호 배타적 토글 제어*
├── 📁 PlacementControlPanel [Anchor: Full-Stretch (0,0)~(1,1)] (PlacementControlUI)
└── 🔘 ToggleStorageButton [Anchor: Bottom-Left (0,0), Pos:(81.5, 0)] (Button, TMP)
```

4. 종합 평가 및 강점 / 취약점 분석 (SWOT)

주요 강점 (Strengths)	잠재적 취약점 및 개선 영역 (Vulnerabilities)
1. 빈틈없는 Safe Area 격리 모든 주요 인터랙션 UI가 `SafeArea`의 직계 자손으로 편성되어 있어 노치, 펀치홀, 다이내믹 아일랜드에 버튼이 가려질 위험 제로.	**1. HUD 개별 요소의 절대 좌표 하드코딩** Gold, Score, Life, Phase가 `Top-Left` 기준 개별 PosX(15, 45.3, 77.4, 108.8)로 배치됨. 텍스트 길이가 늘어나면 옆 텍스트와 겹칠 위험 존재.
2. 스케일 축소로 인한 잘림 원천 차단 CanvasScaler의 `Expand` 모드 채택으로 어떤 비율에서도 캔버스 크기가 540x960 미만으로 줄어들지 않아 외곽 잘림 방지.	**2. 에디터 수동 조작 흔적 잔존** `ObjectStoragePanel`의 Pos(-3, 133), Size(447.33, 276.5) 등 에디터 씬 뷰에서 드래그하여 생긴 소수점 좌표 존재.
3. 동적 컨텐츠의 유연한 자동 배열 슬롯 컨테이너마다 `HorizontalLayoutGroup`과 `GridLayoutGroup`을 적용해 가변 데이터 생성에도 레이아웃 안정성 유지.	**3. 극단적 태블릿 화면비(4:3 등) 대응 제한** `ShopPanel`의 세로 높이(0.2 비율)가 가로로 매우 넓은 태블릿에서는 상대적으로 너무 높게 렌더링될 수 있음.

5. 실무 권장 개선안 (Recommendations)

1. **상단 HUD의 LayoutGroup 화(化)**:
   현재 독립된 PosX를 사용하는 `Gold`, `Score`, `Life`, `Phase`를 하나의 빈 컨테이너(HUDContainer)로 묶고 `HorizontalLayoutGroup`(Spacing: 10, Child Force Expand Width: false)을 적용하면, 숫자가 99,999+ 등으로 늘어나도 텍스트가 겹치지 않고 자연스럽게 밀려납니다.

2. **태블릿/폴더블용 AspectRatioFitter 도입**:
   가로비가 넓은 기기(iPad, Z Fold 펼침)에서 슬롯 패널이 지나치게 커지는 것을 제어하기 위해 메인 패널에 `AspectRatioFitter` 또는 `LayoutElement`의 min/max width 제약을 추가하는 것을 권장합니다.
3. **RectTransform 정수화(Pixel Snapping)**:
   `ObjectStoragePanel` 등 소수점 단위 오프셋이 적용된 UI 요소를 정수 픽셀(예: -3 → 0, 447.33 → 450)로 스냅하여 미세한 서브픽셀 텍스처 번짐 현상을 방지합니다.

6. 결론 (Conclusion)

`TestScene`의 UI 시스템은 **전역 스케일러(CanvasScaler Expand) → 하드웨어 컷아웃 보정(SafeArea.cs) → 기능별 앵커/피벗 분할 → 자동 레이아웃(Layout Groups) → 텍스트 오토사이징(TMP)**의 5단계 방어 체계를 정석적으로 준수하여 구현되었습니다.

모바일 세로형 게임에서 발생할 수 있는 주요 시각적 파손(노치 가림, 슬롯 겹침, 해상도 클리핑, 텍스트 오버플로우)에 대해 높은 내성을 갖추고 있으며, 제안된 HUD LayoutGroup 통합 작업 등을 추가 적용할 경우 상용 배포 수준의 완벽한 멀티 디바이스 반응형 UI를 달성할 수 있습니다.